OFFSIDE LABS

Meteora
DAMM v2 0.1.5

Smart Contract Security Assessment

October 2025

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie

Ronny Xing

Contents

1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs

2 Executive Summary

Introduction

Offside Labs completed a security audit of *DAMM v2* smart contracts, starting on July 16th, 2025, and concluding on September 17th, 2025.

Project Overview

DAMM v2 is a next-generation AMM with enhanced flexibility and efficiency. It supports concentrated liquidity within a custom price range, allowing better capital use. It also expands token support to both SPL and Token 2022 standards.

In Release 0.1.5, DAMM v2 introduces two new swap modes (ExactOut and PartialFill), implements an innovative base fee mode (Rate Limiter) with corresponding trading restrictions, adds a new event format that provides more detailed information, and relaxes certain constraints associated with the Token-2022 Transfer Hook extension.

Audit Scope

The assessment scope contains mainly the smart contracts of the cp-amm program for the *DAMM v2* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- DAMM v2
  - Codebase: `https://github.com/MeteoraAg/damm-v2`
  - Release 0.1.5: `PR-122`
  - Commit Hash: 122677027a44afbb0ffb4faf9e36cdb44a71a542

We listed the PRs we have audited below:

- DAMM v2 PR-54
  - Commit Hash: 7751b1c4964f5f1217ea680b6ca68a7b85e45dd5
- DAMM v2 PR-94
  - Commit Hash: 1a39d2d8d41f8d2aea58c1ded8d8199628c92c50
- DAMM v2 PR-106
  - Commit Hash: 42fb3cb5907e413193312a5b55b31b6b683a2d8f
- DAMM v2 PR-108
  - Commit Hash: 773ae7808f7b17ce2aabf666fea28c50e49b2e18

We listed the files we have audited below:

- DAMM v2 Release 0.1.5:
  - programs/cp-amm/src/base_fee/fee_rate_limiter.rs
  - programs/cp-amm/src/base_fee/fee_scheduler.rs
  - programs/cp-amm/src/base_fee/mod.rs

- programs/cp-amm/src/curve.rs
- programs/cp-amm/src/instructions/admin/ix_create_static_config.rs
- programs/cp-amm/src/instructions/initialize_pool/ix_initialize_customizable_-pool.rs
- programs/cp-amm/src/instructions/initialize_pool/ix_initialize_pool.rs
- programs/cp-amm/src/instructions/ix_add_liquidity.rs
- programs/cp-amm/src/instructions/ix_remove_liquidity.rs
- programs/cp-amm/src/instructions/ix_swap.rs
- programs/cp-amm/src/instructions/swap/ix_swap.rs
- programs/cp-amm/src/instructions/swap/mod.rs
- programs/cp-amm/src/instructions/swap/swap_exact_in.rs
- programs/cp-amm/src/instructions/swap/swap_exact_out.rs
- programs/cp-amm/src/instructions/swap/swap_partial_fill.rs
- programs/cp-amm/src/lib.rs
- programs/cp-amm/src/math/safe_math.rs
- programs/cp-amm/src/math/utils_math.rs
- programs/cp-amm/src/params/fee_parameters.rs
- programs/cp-amm/src/state/config.rs
- programs/cp-amm/src/state/fee.rs
- programs/cp-amm/src/state/pool.rs
- programs/cp-amm/src/utils/token.rs

Findings

The security audit revealed:

- 1 critical issue
- 0 high issue
- 0 medium issue
- 0 low issue
- 3 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	Incorrect amount_in May Lead to Output Drain in ExactIn & PartialFill Mode	Critical	Fixed
02	Insufficient Checks on FeeRateLimiter.max_fee_bps	Informational	Fixed
03	Incorrect Error Type on CollectFeeMode Type Conversion	Informational	Fixed
04	Inconsistent max_fee_numerator Validation Between Pool Creation and Swaps	Informational	Fixed

4 Key Findings and Recommendations

4.1 Incorrect amount_in May Lead to Output Drain in ExactIn & PartialFill Mode

Severity: Critical	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

In `exact_in` mode swaps, the program first checks whether the `amount_in` (after subtracting the Token-2022 transfer fee) is greater than 0, and then calls `pool.get_swap_result_from_exact_input` to calculate the `swap_result` .

```
22      let excluded_transfer_fee_amount_in =
23          calculate_transfer_fee_excluded_amount(token_in_mint,
    ↪ amount_in)?.amount;
24
25      require!(excluded_transfer_fee_amount_in > 0,
        ↪ PoolError::AmountIsZero);
26
27      let swap_result =
28          pool.get_swap_result_from_exact_input(amount_in, fee_mode,
    ↪ trade_direction, current_point)?;
```

programs/cp-amm/src/instructions/swap/swap_exact_in.rs#L22-L28

The `excluded_transfer_fee_amount_in` here represents the actual amount of `input_mint` that enters the pool after the transfer fee deduction. However, when calculating the `swap_result` , the program uses `amount_in` , which still includes the transfer fee.

This causes the input amount used in the `swap_result` calculation to be larger than the actual value transferred into the pool vault.

A similar issue also exists in the `process_swap_partial_fill` function under `PartialFill` mode.

Impact

When the `input_mint` has Token-2022 TransferFee enabled, this issue causes the input amount used in the swap calculation to be larger than the expected value. As a result, the calculated `output_amount` is also inflated, leading the pool to transfer more `output_mint` to the user. If an attacker repeatedly exploits this vulnerability, it could potentially drain almost all of the pool's `output_mint` .

Recommendations

It is recommended to change the `amount_in` parameter passed to `pool.get_swap_result_from_exact_input` and `pool.get_swap_result_from_partial_input` to use `excluded_transfer_fee_amount_in` instead.

Mitigation Review Log

Fixed in the commit 959ce7a6adae9f8e222920cd70d6f94e734ffef3.

4.2 Informational and Undetermined Issues

Insufficient Checks on FeeRateLimiter.max_fee_bps

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Data Validation

In the `RateLimiter` mode, the validation of the `max_fee_bps` field is insufficient in the following three areas:

- In `FeeRateLimiter.validate`, there is not enough validation for `FeeRateLimiter.max_fee_bps`. For example, it does not check whether it exceeds `MAX_FEE_BPS_V1`. Although the outermost layer enforces an upper limit through `MAX_FEE_NUMERATOR_V*`, adding a check here can help prevent abnormal intermediate values caused by an excessively large `max_fee_bps`.
- In `is_zero_rate_limiter`, there is no requirement that `FeeRateLimiter.max_fee_bps` must be 0.
- In `is_non_zero_rate_limiter`, there is no requirement that `FeeRateLimiter.max_fee_bps` must be non-zero.

Fixed in the PR-85 with commit: 851954026422641a6aa000b4e1e53544940cd0b5 and PR-89 with commit: 7a24e556360dec0fb0b7e8f32849f8c5f0ad7d0f.

Incorrect Error Type on CollectFeeMode Type Conversion

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

In the `create_static_config` IX, the `collect_fee_mode` parameter is converted to `CollectFeeMode` using `try_from`. If this conversion fails, the program returns `PoolError::TypeCastFailed`. However, in other parts of the program, a failed conversion to this type returns `PoolError::InvalidCollectFeeMode`. It is recommended to ensure consistency in error handling across the program when using `CollectFeeMode::try_from`.

Fixed in the commit d5366a15b915f10236c0041730f1b39e4fa8313f.

Inconsistent max_fee_numerator Validation Between Pool Creation and Swaps

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Logic Error

The pool version is currently hardcoded as `PoolVersion::V0`. However, when creator tries to create a customized_pool or dynamic_config_pool, they can provide fee config parameters that satisfy the constraint `MAX_FEE_NUMERATOR_V0 < max_fee_numerator <= MAX_FEE_NUMERATOR_V1`. In that case, the BaseFeeHandler validation check can still pass. Although the fee_numerator will be capped at `MAX_FEE_NUMERATOR_V*` according to `pool.version` during swaps, this behavior is inconsistent. Specifically, the expected max_fee_numerator checks may differ between config/pool creation validation and swap validation.

It's recommended to add validation to enforce that the `max_fee_numerator` parameter does not exceed `MAX_FEE_NUMERATOR_V0` when the pool version is V0, aligning the creation-time checks with the swap behavior during swaps.

Fixed in the commit 4f0bae4b0ce3d2023728db8f0115d200cb606628.

5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

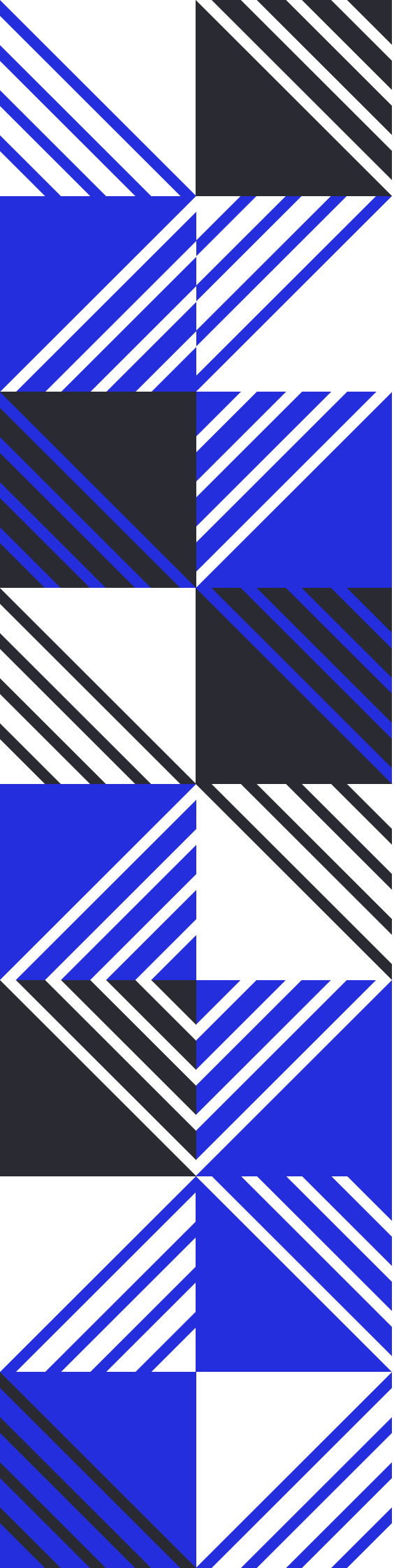
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs